

lab2

April 20, 2026

Given results of observation on capital and labour expenses in some company:

```
[1]: import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
import sympy as sp

df = pd.read_csv("data.csv")
print(df)
```

	K	L	F
0	2860	10680	49920
1	2940	10310	45750
2	2950	10680	50550
3	2880	10800	50570
4	2510	10040	47820
5	2690	10420	47900
6	2990	10940	51900
7	2800	10710	45970
8	3000	9900	48030
9	3070	9930	48100

1 Find multiplicative production function

Multiplicative production function would look like:

$$F(K, L) = AK^\alpha L^\beta$$

Then take log to make it linear:

$$\ln(F(K, L)) = \ln A + \alpha \ln K + \beta \ln L$$

```
[2]: log_K = np.log(df["K"])
log_L = np.log(df["L"])
log_F = np.log(df["F"])

X = np.column_stack((log_K, log_L))
X = sm.add_constant(X)
```

```
model = sm.OLS(log_F, X).fit()
print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          F      R-squared:          0.313
Model:                  OLS    Adj. R-squared:       0.117
Method:                 Least Squares  F-statistic:      1.594
Date:                   Mon, 20 Apr 2026  Prob (F-statistic): 0.269
Time:                   22:19:02  Log-Likelihood:    19.995
No. Observations:       10      AIC:              -33.99
Df Residuals:           7      BIC:              -33.08
Df Model:                2
Covariance Type:        nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	4.2988	3.638	1.182	0.276	-4.304	12.902
x1	0.1519	0.218	0.695	0.509	-0.365	0.668
x2	0.5710	0.358	1.596	0.154	-0.275	1.417

```
=====
Omnibus:                4.878  Durbin-Watson:          2.757
Prob(Omnibus):           0.087  Jarque-Bera (JB):        2.470
Skew:                    -1.217  Prob(JB):                 0.291
Kurtosis:                2.910  Cond. No.                 3.61e+03
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 3.61e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
[3]: log_A = model.params['const']
A = np.exp(log_A)
print("log_A = ", log_A, "; A = ", A)

alpha = model.params['x1']
print("alpha = ", alpha)

beta = model.params['x2']
print("beta = ", beta)
```

```
log_A = 4.298751210451343 ; A = 73.60781561007713
alpha = 0.1519304714689973
beta = 0.5710112130279157
```

$$F = 4.2988K^{0.1519}L^{0.5710}$$

2 Optimal costs of production factors for a firm under conditions of perfect competition in the long and short run

$$\pi(K, L) = pF(K, L) - ((w, r), (L, K)) \rightarrow \max$$

$$\pi(K, L) = p \cdot 4.2988K^{0.1519}L^{0.5710} - wL - rK$$

```
[4]: L_sym, K_sym = sp.symbols('L K', real=True, positive=True)

P_val = 10.0
w_val = 20.0
r_val = 30.0

Q = A * (K_sym**alpha) * (L_sym**beta)
```

2.1 Long run

L and K are variables. The firm chooses a combination where the Marginal Rate of Technical Substitution equals the input price ratio.

```
[5]: profit_lr = P_val * Q - w_val * L_sym - r_val * K_sym

foc_L_lr = sp.diff(profit_lr, L_sym)
foc_K_lr = sp.diff(profit_lr, K_sym)

L_start = float(df["L"].mean())
K_start = float(df["K"].mean())

try:
    res_lr = sp.nsolve([foc_L_lr, foc_K_lr], [L_sym, K_sym], [L_start, K_start], verify=False)
    print(f"LR Opt: L = {float(res_lr[0]):.2f}, K = {float(res_lr[1]):.2f}")
except Exception as e:
    print(f"Exception: {e}", e)
```

LR Opt: L = 22994.65, K = 4078.83

2.2 Short run

One factor K is fixed at level K_0 . Only Labor L is variable.

```
[6]: K0_val = df["K"].mean()
Q_sr = A * (K0_val**alpha) * (L_sym**beta)
profit_sr = P_val * Q_sr - w_val * L_sym - r_val * K0_val

foc_sr = sp.diff(profit_sr, L_sym)
```

```

L_guess = df["L"].mean()

try:
    opt_L_sr_num = sp.nsolve(foc_sr, L_sym, L_guess)
    print(f"Optimal labor input in the Short Run (K={K0_val:.2f}): L = ␣
↪{float(opt_L_sr_num):.2f}")
except Exception as e:
    print(f"Exception: {e}", e)

```

Optimal labor input in the Short Run (K=2869.00): L = 20300.62

3 Monopoly-Monopsony Conditions

In this scenario, the firm is the sole seller (Price P depends on Q) and the sole buyer of resources (Prices w, r depend on L, K).

Market Assumptions: 1. Demand for product: $P(Q) = a - bQ$ 2. Resource supply: $w(L) = w_0 + cL$ and $r(K) = r_0 + dK$

```

[7]: a_m = 27.0
     b_m = 0.0001

     # Monopsony parameters (resource prices increase as demand grows)
     w0_m = 10.0 # Base wage (intercept for labor supply)
     c_m = 0.0005 # Wage slope (marginal increase in w per +1 unit of L)
     r0_m = 15.0 # Base rent
     d_m = 0.001 # Rent slope

     # Defining functions using your estimated A, alpha, and beta
     price_func = a_m - b_m * Q
     wage_func = w0_m + c_m * L_sym
     rent_func = r0_m + d_m * K_sym

     profit_mono = (price_func * Q) - (wage_func * L_sym + rent_func * K_sym)

     # First-Order Conditions: partial derivatives with respect to L and K
     foc_L_mono = sp.diff(profit_mono, L_sym)
     foc_K_mono = sp.diff(profit_mono, K_sym)

     try:
         # LR results [22994, 4078] as the initial guess for nsolve
         opt_mono = sp.nsolve([foc_L_mono, foc_K_mono], [L_sym, K_sym], [res_lr[0], ␣
↪res_lr[1]], verify=False)

         L_m, K_m = float(opt_mono[0]), float(opt_mono[1])
         Q_m = float(Q.subs({L_sym: L_m, K_sym: K_m}))
         P_m = a_m - b_m * Q_m

```

```

w_m = w0_m + c_m * L_m
r_m = r0_m + d_m * K_m

print(f"Optimal factor inputs: L = {L_m:.2f}, K = {K_m:.2f}")
print(f"Resource prices: w = {w_m:.2f}, r = {r_m:.2f}")
print(f"Production output: Q = {Q_m:.2f}")
print(f"Product price: P = {P_m:.2f}")

except Exception as e:
    print(f"Error: {e}. Try increasing a_m or decreasing b_m slightly.")

```

Optimal factor inputs: L = 18262.07, K = 5345.37

Resource prices: w = 19.13, r = 20.35

Production output: Q = 73571.60

Product price: P = 19.64